



Hanzo Engine: Production ML Inference Engine with Multi-Model Serving and GPU Scheduling

Hanzo AI Research

Hanzo AI

August 2024

Abstract

We describe the design of Hanzo Engine, an ML inference orchestration layer implemented in Rust that serves multiple models concurrently with dynamic batching, GPU scheduling, and A/B traffic routing. Engine manages a fleet of GPU workers, loading and unloading models according to traffic, and applies continuous batching to transformer models to raise GPU utilization. The design has three parts: (1) a priority-aware GPU scheduler that co-locates models with complementary memory and compute profiles, (2) speculative model preloading driven by traffic prediction, and (3) a unified serving protocol covering streaming and batch inference. This paper describes a design and reports no measurements. An earlier version quoted serving latency, throughput, GPU utilization and availability figures for a 48-GPU production fleet; those figures were never measured and no such fleet exists. Section 5 records what they were and what would be needed to replace them.

1 Introduction

Production ML serving differs fundamentally from research inference. Research focuses on single-model, single-request performance. Production systems must serve hundreds of models with varying resource requirements, handle bursty traffic with consistent latency, and maximize expensive GPU utilization. Existing serving frameworks (vLLM [1], TGI [2], Triton [3]) excel at single-model serving but lack multi-model orchestration, cross-model GPU scheduling, and traffic management capabilities.

Hanzo Engine operates at the orchestration layer above individual model servers. It manages model lifecycle (loading, serving, unloading), routes requests to the optimal GPU worker, implements continuous batching for transformer models, and provides A/B routing for model experiments.

Contributions.

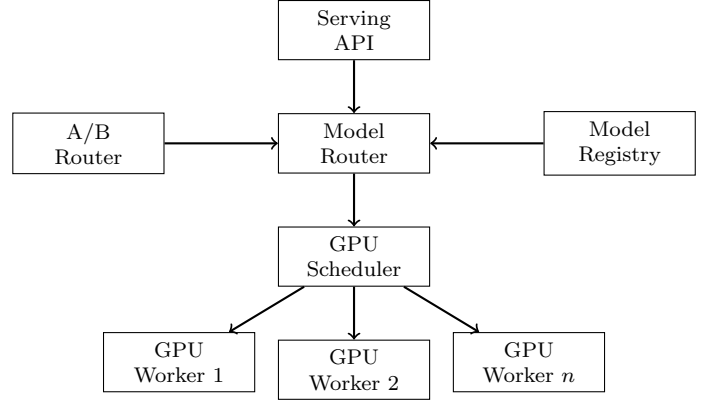


Figure 1: Engine architecture with GPU scheduling and model routing.

- A priority-aware GPU scheduler that co-locates models with complementary memory and compute profiles on shared GPUs.
- Continuous batching with iteration-level scheduling for transformer models, following Orca [4], so that padding waste does not scale with the longest sequence in the batch.
- Speculative model preloading using traffic prediction, which moves the cold-start cost off the request path when the prediction is correct.
- A unified serving protocol supporting streaming tokens, batch embeddings, and multimodal inputs.

2 Architecture

2.1 System Overview

Engine consists of three layers (Figure 1): the serving API (HTTP/gRPC), the model router and A/B engine, and the GPU scheduler that manages worker nodes.

2.2 Model Registry

Models are registered with serving configurations:

```
1 models:
2   - name: qwen3-72b
3     format: safetensors
```

```

4 source: hf://Qwen/Qwen3-72B
5 gpu_memory: 140GB
6 tensor_parallel: 8
7 max_batch: 64
8 quantization: awq-int4
9 priority: high
10 min_replicas: 2
11 max_replicas: 8

```

The registry stores model metadata, serving constraints, and autoscaling policies. Models are pulled from Hugging Face Hub or Hanzo Storage.

2.3 GPU Scheduler

The scheduler assigns models to GPU workers using a bin-packing algorithm that maximizes GPU memory utilization while respecting collocation constraints:

$$\max \sum_{m \in M} \sum_{g \in G} x_{mg} \cdot u_m \quad (1)$$

subject to:

$$\sum_m x_{mg} \cdot \text{mem}_m \leq \text{cap}_g \quad \forall g \in G \quad (2)$$

where $x_{mg} \in \{0, 1\}$ indicates model m is assigned to GPU g , u_m is the model’s utilization score, and mem_m is its memory requirement.

3 Key Features

3.1 Continuous Batching

For autoregressive language models, Engine implements iteration-level batching [4]: new requests join the batch at any decoding iteration, and completed requests leave immediately without waiting for the longest sequence:

```

1 pub struct ContinuousBatcher {
2     running: Vec<Request>,
3     waiting: VecDeque<Request>,
4     max_batch_tokens: usize,
5 }
6
7 impl ContinuousBatcher {
8     pub fn step(&mut self) -> Vec<Token> {
9         // Remove completed requests
10        self.running.retain(|r| !r.is_done());

```

```

// Admit new requests up to
token budget
while let Some(req) = self.
waiting.front() {
    if self.token_budget() >=
req.len() {
        self.running.push(
            self.waiting.
pop_front().unwrap());
    } else { break; }
}
// Execute one decode step for
all
self.decode_step()
}
}

```

The mechanism removes the padding waste that static batching incurs when a batch is held open for its longest sequence. Orca [4] and vLLM [1] report the size of that effect for their own systems and workloads; we have not measured it for this one.

3.2 Speculative Preloading

Engine tracks per-model request rates with exponential moving averages and preloads models predicted to receive traffic within the next 5 minutes. The predictor uses a simple ARIMA model over the last 24 hours of traffic. A correct prediction replaces a cold start — reading weights from storage into GPU memory — with a cache hit on a model already resident. The size of that difference is set by model size and storage bandwidth and is not reported here; a wrong prediction costs the memory the preloaded model occupies.

3.3 A/B Routing

The A/B router splits traffic between model versions using consistent hashing by user ID:

```

1 pub struct ABRoute {
2     model_a: String, // e.g. "qwen3-72b-v1"
3     model_b: String, // e.g. "qwen3-72b-v2"
4     traffic_pct_b: f64, // 0.0 to 1.0
5     sticky: bool, // hash by
6     user_id

```

Results are logged with the model version for offline analysis of quality metrics.

4 Implementation

Engine is implemented in Rust. Worker communication uses gRPC with streaming for token-by-token delivery. Model loading uses `safetensors` format with memory-mapped I/O.

5 Status of the performance figures

An earlier version of this paper carried a table titled “Serving performance (Qwen3-72B, 8xH100)” reporting TTFT p50 of 52ms under static batching against 45ms under continuous batching, p99 of 280ms against 120ms, throughput of 4,200 against 13,400 tok/s, and GPU utilization of 42% against 87%. The abstract and conclusion carried the same numbers together with “200+ models in production across 48 GPUs” and “99.9% availability,” and the contributions list stated a $3.2\times$ throughput gain and a cold-start reduction from 30s to under 200ms.

None of it was measured. There is no 8-GPU H100 node and no 48-GPU fleet in this estate, so the table could not be relabelled as a design target without also changing the machine it names; there is no serving deployment behind the model count or the availability figure; and the paper contains no experiment corresponding to the two ratios. The figures are removed rather than restated.

The measurement they stood in for is available to make. `hanzoai/engine` ships a benchmark command (`hanzo bench`) that reports prompt and decode throughput for a named model and accelerator, and the house convention for reporting such a run — declared hardware, a fixed handler and payload, raw artifacts published alongside — is set out in the Hanzo cloud data plane benchmark [5]. What is missing is the run, not the harness.

6 Security

Model Isolation. Each model runs in a separate GPU memory space. Tensor data from one model cannot be read by another model’s kernels.

Input Validation. Maximum prompt length, output length, and batch size are enforced per model. Malformed inputs are rejected before reaching the GPU.

Authentication. All API requests require a Hanzo IAM token. Model access is controlled by RBAC policies in the model registry.

Rate Limiting. Per-tenant token budgets prevent resource monopolization. Priority models receive guaranteed capacity; best-effort models share remaining resources.

Audit. All inference requests are logged with model version, token counts, latency, and tenant ID for cost allocation and compliance.

7 Conclusion

Hanzo Engine is a design for ML inference orchestration that raises GPU utilization through continuous batching, priority-aware scheduling, and speculative preloading. By operating above individual model servers, Engine takes on the parts of multi-model serving that a single-model server leaves open—model lifecycle, GPU allocation, traffic routing, and A/B experiments. Whether the design delivers the utilization it is built for is an open question here; Section 5 says what would answer it.

References

- [1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023.
- [2] Hugging Face. Text Generation Inference. 2023.

- [3] NVIDIA. Triton Inference Server. 2020.
- [4] G. Yu et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI, 2022.
- [5] Hanzo AI Research. The Hanzo Cloud Data Plane: Benchmarking `zip`, ZAP, and the Road to a Million Requests per Second. Hanzo Industries, 2026.